

CODIGO MEMORIA FINAL (artemis_3)

¡Perfecto! Aquí tienes un paquete completo, listo para copiar y pegar directamente en tu editor, que implementa la Opción A de forma funcional, ejecutable y con un MVP operativo (persistencia episódica/semántica, ChromaDB vectorial, retrieval híbrido con scoring, consolidación/aprendizaje, factcheck/evaluación, trazabilidad, políticas, MCP server mínimo con FastAPI, CLI, y modo *demo/offline* sin necesidad de API keys).

✅ *Requisitos: Python 3.10+, pip. Funciona sin OpenAI (usa embeddings deterministas por hash para demo). Si quieres embeddings reales, añade OPENAI_API_KEY en .env.*



Instrucciones (1 minuto para ponerlo en marcha)

1. Crea la carpeta del proyecto:
2. Bash
3. `mkdir -p mi-proyecto && cd mi-proyecto`
4. Copia/pega TODO el árbol de archivos que viene a continuación (cada bloque es un archivo; crea las carpetas si no existen).
5. Crea/activa entorno virtual e instala dependencias:
6. Bash

```
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
```

7. `pip install -r requirements.txt`
8. (Opcional, para embeddings reales) copia `.env.example` a `.env` y completa `OPENAI_API_KEY`.
9. Ejecuta la demo end-to-end:

10. Bash

11. `python make_demo.py`

12. Esto: ingestará 2 episodios, consolidará (episódico→resumen→semántico), indexará en Chroma, hará retrieval, factcheck, evaluación y guardará trazas/logs.

13. (Opcional) Levanta el MCP server (FastAPI):

14. Bash

15. `uvicorn ai_architect.core.mcp_server:app --reload --port 8000`

16. Endpoints: POST /ingest, POST /retrieve, POST /consolidate, POST /evaluate, GET /health.

17. (Opcional) CLI directo:

18. Bash

```
python ingest_cli.py --file memory/episodic/demo_epl.json
python retrieve_cli.py --query "decisión de autenticación" --project "demo-auth" --top_k 3
python consolidate_cli.py
```

19. `python evaluate_cli.py --query "decisión de autenticación" --project "demo-auth"`

Árbol de archivos (copia/pega completo)

1) requirements.txt

txt

```
fastapi==0.111.0
uvicorn[standard]==0.30.1
chromadb==0.5.5
pydantic==2.8.2
pydantic-settings==2.3.4
typer==0.12.3
rich==13.7.1
python-dotenv==1.0.1
```

```
numpy==1.26.4
orjson==3.10.7
tenacity==8.5.0
httpx==0.27.0
```

2) .env.example

env

```
# Embeddings
EMBEDDING_PROVIDER=hash # 'hash' (offline/demo) o 'openai'
OPENAI_API_KEY=sk-xxxx
OPENAI_EMBEDDING_MODEL=text-embedding-3-small

# Chroma
CHROMA_PERSIST_DIR=.ai_memory/chroma

# App
APP_ENV=dev
LOG_DIR=logs
MEMORY_ROOT=memory
AI_DIR=.ai
```

3) opencode.json (opcional, para tooling/agent config)

JSON

```
{
  "name": "ai-knowledge-system",
  "version": "0.1.0",
  "mcp": {
    "url": "http://localhost:8000"
  },
  "capabilities": ["ingest", "retrieve", "consolidate", "evaluate", "factcheck"]
}
```



Documentación / Gobernanza (dentro de .ai/)

4) .ai/system.md

Markdown

```
# Sistema de Memoria y Aprendizaje (Opción A)
```

Objetivos:

- Persistencia de conocimiento (episódico + semántico + resúmenes + reflexiones)
- Recuperación eficiente híbrida (vectorial + metadatos + scoring + filtros)

- Consolidación/aprendizaje (ingest → resumen → reflexión → promoción a semántico → re-index)
- Gobernanza/confianza (políticas, trazabilidad, factcheck, evaluación, logs auditables)
- Compatibilidad multi-modelo (módulos Python reutilizables + MCP server + archivos versionables)
- Separación episódica/semántica explícita

Principios:

1. Toda unidad de conocimiento referencia evidencia (source_ids).
2. Todo retrieval devuelve score + evidencia + trazabilidad (trace_id).
3. Toda promoción a semántico requiere factcheck mínimo (fuente, consistencia, contradicción).
4. Todo evento se audita en logs/traces y logs/evaluations.

5) .ai/rules.md

Markdown

Reglas operativas

- No almacenar PII sensible (passwords, tokens, DNI, tarjetas). Si aparece, enmascarar como [REDACTED].
- Todo episodio debe incluir: id, project, timestamp (ISO8601 UTC), source_type, author, decisions[], evidence[], tags[].
- Todo ítem semántico debe incluir: id, type (fact/decision/pattern/mistake), statement, confidence (0-1), evidence_source_ids[], contradictions[], created_at, updated_at.
- Consolidación: solo promover a semántico si confidence ≥ 0.6 y al menos 1 evidencia verificable.
- Retrieval: siempre devolver top_k con score, filtros aplicados, y trace_id.
- Evaluación: registrar relevance (0-1), has_evidence (bool), hallucination_risk (0-1), notes.

6) .ai/context.md

Markdown

Contexto del proyecto

Este repositorio implementa un sistema de memoria para agentes/ equipos:

- Memoria episódica: eventos/decisiones con evidencia.
- Memoria semántica: hechos/patronos/errores consolidados.
- Working/summaries/reflections/indexes para ciclo de aprendizaje.
- Chroma para vectores; JSON/MD para auditabilidad.
- FastAPI MCP para interoperabilidad multi-modelo.

7) .ai/agents.md

Markdown

Agentes/roles

- Ingestor: transforma entradas (reuniones/PRs/issues/chat) en episodios validados.
- Resumidor: genera resumen estructurado + decisiones + acciones + riesgos.
- Reflector: detecta patrones, contradicciones, lecciones aprendidas.

- Factchecker: valida consistencia con semántica/episódicos, marca contradicciones.
- Consolidador: promueve a semántico con evidencia y política de confianza.
- Retriever: híbrido vector+metadatos+scoring; retorna evidencia.
- Evaluador: mide calidad de retrieval/generación (relevance, evidencia, riesgo).

8) .ai/memory/schema.md

Markdown

```
# Esquema de datos (resumen)

## Episodio (memory/episodic/*.json)
- id (uuid)
- project (string)
- timestamp (ISO8601 UTC)
- source_type (meeting|pr|issue|chat|doc|incident)
- author (string)
- title (string)
- summary (string)
- decisions [ {text, owner, due_date?} ]
- actions [ {text, owner, due_date?} ]
- risks [ {text, severity} ]
- evidence [ {type, url_or_path, excerpt} ]
- tags [string]
- embedding_text (string) # texto canónico para embedding

## Semántico (memory/semantic/*.json)
- id (uuid)
- type (fact|decision|pattern|mistake)
- project (string)
- statement (string)
- confidence (0.0-1.0)
- evidence_source_ids [episode_id]
- contradictions [ {other_semantic_id, reason} ]
- created_at, updated_at
- tags [string]
- embedding_text (string)

## Resumen (memory/summaries/*.json)
- episode_id, summary_text, decisions[], actions[], risks[], created_at

## Reflexión (memory/reflections/*.json)
- id, scope {project?, date_range?}, patterns [string], contradictions [string], hypotheses [string], created_at

## Traza (logs/traces/*.jsonl)
- trace_id, ts, operation (ingest|retrieve|consolidate|evaluate|factcheck), inputs, outputs, model/provider, latency_ms, score, evidence_ids[], errors[]

## Evaluación (logs/evaluations/*.jsonl)
- trace_id, query, retrieved_ids[], relevance[], has_evidence[], hallucination_risk[], notes
```

9) .ai/memory/retrieval.md

Markdown

Estrategia de Retrieval

1. ****Entrada****: query, project (opcional), tags (opcional), date_from/date_to (opcional), top_k, filters.
2. ****Embedding****: query → vector (hash-offline o OpenAI). Misma función que ingest.
3. ****Candidatos****:
 - Chroma: similitud coseno sobre colección `semantic` y/o `episodic` (configurable).
 - Filtros estrictos: project, tags, timestamp rango.
4. ****Scoring híbrido**** (0-1):
 - s_vec = similitud coseno normalizada (0-1)
 - s_recency = 1.0 si < 7 días, 0.8 <30, 0.6 <90, 0.4 else (sobre timestamp)
 - s_evidence = 0.5 + 0.5*(#evidencias/5) (cap 1.0)
 - s_conf = confidence semántico o 0.7 por defecto para episódico
 - score = 0.45*s_vec + 0.2*s_recency + 0.2*s_evidence + 0.15*s_conf
5. ****Salida****: lista ordenada con id, tipo, texto, proyecto, timestamp, score, evidencia[], trace_id.
6. ****Evaluación****: relevance humana/LLM, has_evidence, hallucination_risk; guardar en logs/evaluations.

10) .ai/memory/consolidation.md

Markdown

Política de Consolidación (episódico → semántico)

1. Ingestar episodio válido (schema completo).
2. Generar resumen estructurado (decisiones/acciones/riesgos) → `memory/summaries/`.
3. Reflexión (opcional, batch): detectar patrones/contradicciones → `memory/reflections/`.
4. Factcheck:
 - Buscar hechos/decisiones similares en semántico (misma statement/entidad).
 - Si existe: comparar; si contradice → registrar `contradictions[]` y NO sobrescribir; si apoya → subir `confidence` y añadir `evidence\_source\_ids`.
 - Si no existe: proponer ítem semántico.
5. Promoción: crear `memory/semantic/\*.json` solo si `confidence >= 0.6` y ≥ 1 evidencia (episode_id).
6. Indexar en Chroma colección `semantic` (y opcional `episodic`).
7. Trazabilidad: guardar trace consolidación con lista de source_ids y decisión.

11) .ai/memory/policies.md

Markdown

Políticas de Gobernanza

- ****Retención****: resúmenes/episodios 24 meses; semántico indefinido salvo obsolescencia marcada. Reflections 12 meses.
- ****Acceso****: solo código/servicio interno; no exponer endpoints crudos sin auth en prod.
- ****PII****: redacción obligatoria. Regex: emails, teléfonos, DNI, tarjetas, tokens `sk-`, `Bearer `, claves `AKIA...`. Reemplazar por [REDACTED].
- ****Versionado****: JSON con `id`, `created\_at`, `updated\_at`. Cambios a semántico generan nueva versión (append) y marcan `supersedes`/`superseded\_by` si aplica.
- ****Auditoría****: todo ingest/retrieve/consolidate/evaluate genera `logs/traces/\*.jsonl` con trace_id.
- ****Export/Backup****: `python -m ai\_architect.pipelines.export` genera
`.ai\_memory/exports/YYYY-MM-DD.zip` con
episodic/semantic/summaries/reflections/indexes/meta.

12) .ai/reasoning/factcheck.md

Markdown

```
# Proceso de Factcheck

- Consistencia interna: decisiones coherentes con resumen y evidencia del episodio.
- Consistencia histórica: buscar en semántico ítems con misma entidad/proyecto/tags; detectar contradicciones.
- Verificabilidad: cada afirmación debe tener ≥1 evidencia concreta (url/path/excerpt).
- Confianza: 0-1. Hecho duplicado/apoyado +0.1; contradicción -0.3; evidencia débil -0.2.
- Salida: `ok(bool)`, `confidence`, `matched_semantic_ids[]`, `contradictions[]`, `actions[]` (revisar, dividir, descartar).
```

13) .ai/reasoning/tool_usage.md

Markdown

```
# Uso de herramientas (MCP)

- `ingest(episode_json)` : valida schema, redacta PII, guarda episodic, indexa episodic (opcional), devuelve episode_id/trace_id.
- `retrieve(query, filters)` : devuelve top_k con score/evidencia/trace_id.
- `consolidate(scope)` : corre consolidación (proyecto o global), devuelve resumen de promociones/contradicciones/trace_id.
- `evaluate(query, filters)` : ejecuta retrieve+evaluación, guarda logs/evaluations, devuelve métricas agregadas.
- `factcheck(statement_or_episode_id)` : devuelve veredicto + confianza + evidencias + contradicciones.
```

14) .ai/reasoning/evaluation.md

Markdown

```
# Evaluación
```

Métricas por retrieve:

- **Relevance** (0-1): ¿responde la intención de la query?
- **Has_evidence** (bool): ¿incluye evidencia verificable?
- **Hallucination_risk** (0-1): contradicciones, afirmaciones sin fuente, ambigüedad.
- **Recall@k / Precision@k** (cuando hay ground truth).
- **Time-to-answer** (latencia) y **coste** (si embeddings pagas).

Registro: `logs/evaluations/\*.jsonl` con `trace\_id`, `query`, `retrieved\_ids`, `scores`, `relevance`, `has\_evidence`, `hallucination\_risk`, `notes`.

15) .ai/knowledge/architecture.md

Markdown

Arquitectura

- ``ai_architect/core/``: dominio (memory, retrieval, consolidation, scoring, router, embeddings, redact, utils, mcp_server)
- ``ai_architect/pipelines/``: ingest, summarize, reflect, factcheck, evaluate, export
- ``memory/``: datos (episodic/, semantic/, working/, summaries/, reflections/, indexes/)
- ``.ai_memory/chroma/``: vector store persistente
- ``logs/``: trazas, tool_calls, evaluations, failures
- ``.ai/``: documentación/políticas/reglas (single source of truth operativo)



Código Python (core + pipelines + CLI + server)

Nota: todo es autocontenido. El módulo `embeddings.py` soporta `EMBEDDING_PROVIDER=hash` (sin red) y `openai` (si configuras API key).

16) `ai_architect/__init__.py`

Python

```
__version__ = "0.1.0"
```

17) `ai_architect/core/utils.py`

Python

```
import re, json, uuid, hashlib, datetime as dt
from pathlib import Path
from typing import Any, Dict

ISO = "%Y-%m-%dT%H:%M:%SZ"

PII_PATTERNS = [
    re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b"),      # email
    re.compile(r"\b(?:\+?\d[\d\s\-\]{7,}\d)\b"),                        # phone
    re.compile(r"\b\d{6,}\b"),                                           # long
    numbers (DNI/tarjeta simplificado)
    re.compile(r"\bsk-[A-Za-z0-9]{16,}\b"),                             #
    OpenAI-like keys
    re.compile(r"\bBearer\s+[A-Za-z0-9\-\.\_~+/=]{20,}\b", re.IGNORECASE), # Bearer
    tokens
    re.compile(r"\bAKIA[0-9A-Z]{16}\b"),                                # AWS key id
    (simplificado)
]

def now_iso() -> str:
    return dt.datetime.utcnow().strftime(ISO)

def new_id(prefix: str = "") -> str:
    return f"{prefix}{uuid.uuid4().hex}"

def redact_text(text: str) -> str:
```



```

    red = text
    for pat in PII_PATTERNS:
        red = pat.sub("[REDACTED]", red)
    return red

def ensure_dir(p: Path) -> Path:
    p.mkdir(parents=True, exist_ok=True)
    return p

def write_json(p: Path, data: Dict[str, Any]) -> None:
    ensure_dir(p.parent)
    with open(p, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2, sort_keys=True)

def read_json(p: Path) -> Dict[str, Any]:
    with open(p, "r", encoding="utf-8") as f:
        return json.load(f)

def append_jsonl(p: Path, data: Dict[str, Any]) -> None:
    ensure_dir(p.parent)
    with open(p, "a", encoding="utf-8") as f:
        f.write(json.dumps(data, ensure_ascii=False) + "\n")

def hash_text(s: str) -> str:
    return hashlib.sha256(s.encode("utf-8")).hexdigest()

def date_range_filter_ok(ts_iso: str, date_from: str | None, date_to: str | None) -> bool:
    try:
        t = dt.datetime.strptime(ts_iso, ISO)
        if date_from:
            df = dt.datetime.strptime(date_from, ISO)
            if t < df:
                return False
        if date_to:
            dtto = dt.datetime.strptime(date_to, ISO)
            if t > dtto:
                return False
        return True
    except Exception:
        return True

```

18) ai_architect/core/config.py

Python

```

from pathlib import Path
from pydantic_settings import BaseSettings
from dotenv import load_dotenv

load_dotenv()

class Settings(BaseSettings):
    app_env: str = "dev"
    embedding_provider: str = "hash" # hash | openai
    openai_api_key: str | None = None
    openai_embedding_model: str = "text-embedding-3-small"
    chroma_persist_dir: str = ".ai_memory/chroma"
    log_dir: str = "logs"
    memory_root: str = "memory"
    ai_dir: str = ".ai"

```

```

class Config:
    env_file = ".env"
    case_sensitive = False

settings = Settings()

ROOT = Path.cwd()
AI_DIR = ROOT / settings.ai_dir
MEMORY_ROOT = ROOT / settings.memory_root
CHROMA_DIR = ROOT / settings.chroma_persist_dir
LOG_DIR = ROOT / settings.log_dir

# Memory subdirs
EPISODIC_DIR = MEMORY_ROOT / "episodic"
SEMANTIC_DIR = MEMORY_ROOT / "semantic"
WORKING_DIR = MEMORY_ROOT / "working"
SUMMARIES_DIR = MEMORY_ROOT / "summaries"
REFLECTIONS_DIR = MEMORY_ROOT / "reflections"
INDEXES_DIR = MEMORY_ROOT / "indexes"

# Logs subdirs
TRACES_DIR = LOG_DIR / "traces"
TOOLCALLS_DIR = LOG_DIR / "tool_calls"
EVALS_DIR = LOG_DIR / "evaluations"
FAILS_DIR = LOG_DIR / "failures"

for p in [AI_DIR, MEMORY_ROOT, CHROMA_DIR, LOG_DIR,
          EPISODIC_DIR, SEMANTIC_DIR, WORKING_DIR, SUMMARIES_DIR, REFLECTIONS_DIR,
          INDEXES_DIR,
          TRACES_DIR, TOOLCALLS_DIR, EVALS_DIR, FAILS_DIR]:
    p.mkdir(parents=True, exist_ok=True)

```

19) ai_architect/core/embeddings.py

Python

```

import hashlib
import numpy as np
from typing import List
from .config import settings

def _hash_embed(text: str, dim: int = 1536) -> List[float]:
    # Deterministic, offline-friendly embedding for demo/testing
    h = hashlib.sha256(text.encode("utf-8")).digest()
    # Expand to dim using repeated hash
    out = []
    seed = h
    while len(out) < dim:
        seed = hashlib.sha256(seed).digest()
        for b in seed:
            out.append((b / 255.0) * 2.0 - 1.0) # [-1,1]
            if len(out) >= dim:
                break
    return out[:dim]

def embed_texts(texts: List[str]) -> List[List[float]]:
    if settings.embedding_provider == "openai":
        try:
            from openai import OpenAI

```

```

        client = OpenAI(api_key=settings.openai_api_key)
        resp = client.embeddings.create(model=settings.openai_embedding_model,
input=texts)
        return [d.embedding for d in resp.data]
    except Exception as e:
        # Fallback to hash if OpenAI fails
        return [_hash_embed(t) for t in texts]
else:
    return [_hash_embed(t) for t in texts]

def embed_text(text: str) -> List[float]:
    return embed_texts([text])[0]

```

20) ai_architect/core/redact.py

Python

```

from .utils import redact_text

def redact_episode(episode: dict) -> dict:
    ep = dict(episode)
    ep["title"] = redact_text(ep.get("title", ""))
    ep["summary"] = redact_text(ep.get("summary", ""))
    ep["decisions"] = [
        {**d, "text": redact_text(d.get("text", ""))} for d in ep.get("decisions", [])
    ]
    ep["actions"] = [
        {**a, "text": redact_text(a.get("text", ""))} for a in ep.get("actions", [])
    ]
    ep["risks"] = [
        {**r, "text": redact_text(r.get("text", ""))} for r in ep.get("risks", [])
    ]
    ep["evidence"] = [
        {**e, "excerpt": redact_text(e.get("excerpt", ""))} for e in ep.get("evidence", [])
    ]
    ep["embedding_text"] = redact_text(ep.get("embedding_text", ""))
    return ep

```

21) ai_architect/core/memory.py

Python

```

from pathlib import Path
from typing import Dict, Any, List
from .config import EPISODIC_DIR, SEMANTIC_DIR, SUMMARIES_DIR, REFLECTIONS_DIR
from .utils import write_json, read_json, now_iso, new_id, ensure_dir

def save_episode(episode: Dict[str, Any]) -> Path:
    if "id" not in episode:
        episode["id"] = new_id("ep_")
    if "created_at" not in episode:
        episode["created_at"] = now_iso()
    p = EPISODIC_DIR / f"{episode['id']}.json"
    write_json(p, episode)
    return p

def load_episode(episode_id: str) -> Dict[str, Any]:

```

```

p = EPISODIC_DIR / f"{episode_id}.json"
return read_json(p)

def list_episodes(project: str | None = None) -> List[Dict[str, Any]]:
    items = []
    for p in EPISODIC_DIR.glob("*.json"):
        try:
            d = read_json(p)
            if project and d.get("project") != project:
                continue
            items.append(d)
        except Exception:
            continue
    return items

def save_summary(summary: Dict[str, Any]) -> Path:
    if "id" not in summary:
        summary["id"] = new_id("sum_")
    if "created_at" not in summary:
        summary["created_at"] = now_iso()
    p = SUMMARIES_DIR / f"{summary['id']}.json"
    write_json(p, summary)
    return p

def save_semantic(item: Dict[str, Any]) -> Path:
    if "id" not in item:
        item["id"] = new_id("sem_")
    if "created_at" not in item:
        item["created_at"] = now_iso()
    if "updated_at" not in item:
        item["updated_at"] = now_iso()
    p = SEMANTIC_DIR / f"{item['id']}.json"
    write_json(p, item)
    return p

def load_semantic(semantic_id: str) -> Dict[str, Any]:
    p = SEMANTIC_DIR / f"{semantic_id}.json"
    return read_json(p)

def list_semantic(project: str | None = None, type_: str | None = None) -> List[Dict[str, Any]]:
    items = []
    for p in SEMANTIC_DIR.glob("*.json"):
        try:
            d = read_json(p)
            if project and d.get("project") != project:
                continue
            if type_ and d.get("type") != type_:
                continue
            items.append(d)
        except Exception:
            continue
    return items

def save_reflection(ref: Dict[str, Any]) -> Path:
    if "id" not in ref:
        ref["id"] = new_id("ref_")
    if "created_at" not in ref:
        ref["created_at"] = now_iso()
    p = REFLECTIONS_DIR / f"{ref['id']}.json"
    write_json(p, ref)
    return p

```

22) ai_architect/core/vectorstore.py

Python

```
import chromadb
from chromadb.utils import embedding_functions
from typing import List, Dict, Any
from .config import CHROMA_DIR, settings
from .embeddings import embed_texts, embed_text

COL_EPISODIC = "episodic"
COL_SEMANTIC = "semantic"

_client = None

def get_client():
    global _client
    if _client is None:
        _client = chromadb.PersistentClient(path=str(CHROMA_DIR))
    return _client

def _collection(name: str):
    client = get_client()
    # Use default EF (we generate embeddings ourselves) - pass explicit none not required;
    # we provide embeddings
    return client.get_or_create_collection(name=name, metadata={"hnsw:space": "cosine"})

def index_episode(episode: Dict[str, Any]) -> None:
    col = _collection(COL_EPISODIC)
    emb_text = episode.get("embedding_text") or episode.get("summary") or episode.get("title") or ""
    vec = embed_text(emb_text)
    meta = {
        "id": episode.get("id"),
        "project": episode.get("project", ""),
        "timestamp": episode.get("timestamp", ""),
        "source_type": episode.get("source_type", ""),
        "tags": ", ".join(episode.get("tags", [])),
    }
    col.upsert(ids=[episode["id"]], embeddings=[vec], metadatas=[meta], documents=[emb_text])

def index_semantic(item: Dict[str, Any]) -> None:
    col = _collection(COL_SEMANTIC)
    emb_text = item.get("embedding_text") or item.get("statement") or ""
    vec = embed_text(emb_text)
    meta = {
        "id": item.get("id"),
        "project": item.get("project", ""),
        "type": item.get("type", ""),
        "timestamp": item.get("updated_at") or item.get("created_at", ""),
        "tags": ", ".join(item.get("tags", [])),
        "confidence": float(item.get("confidence", 0.7)),
    }
    col.upsert(ids=[item["id"]], embeddings=[vec], metadatas=[meta], documents=[emb_text])

def query_collection(collection: str, query: str, n_results: int = 5, where: Dict[str, Any] | None = None) -> Dict[str, Any]:
    col = _collection(collection)
    vec = embed_text(query)
```

```

res = col.query(query_embeddings=[vec], n_results=n_results, where=where,
include=["metadatas", "documents", "distances", "ids"])
# Normalize
out = {"ids": res.get("ids", [[]])[0], "metadatas": res.get("metadatas", [[]])[0],
"documents": res.get("documents", [[]])[0], "distances": res.get("distances", [[]])[0]}
return out

```

23) ai_architect/core/scoring.py

Python

```

import datetime as dt
from typing import Dict, Any
from .utils import ISO

def _recency_score(ts_iso: str) -> float:
    try:
        t = dt.datetime.strptime(ts_iso, ISO)
        now = dt.datetime.utcnow()
        delta = now - t
        days = delta.days
        if days < 7: return 1.0
        if days < 30: return 0.8
        if days < 90: return 0.6
        if days < 365: return 0.4
        return 0.2
    except Exception:
        return 0.5

def _evidence_score(item: Dict[str, Any]) -> float:
    ev = item.get("evidence", [])
    if not ev:
        # semantic evidence_source_ids
        ev = item.get("evidence_source_ids", [])
    n = len(ev)
    s = 0.5 + 0.5 * (n / 5.0)
    if s > 1.0: s = 1.0
    if s < 0.0: s = 0.0
    return s

def _confidence_score(item: Dict[str, Any]) -> float:
    c = item.get("confidence", None)
    if c is None:
        # episodic default
        return 0.7
    try:
        return float(c)
    except Exception:
        return 0.7

def cosine_from_distance(dist: float) -> float:
    # Chroma cosine distance: 0 = identical, 2 = opposite; cosine similarity ~ 1 - dist/2
    sim = 1.0 - (dist / 2.0)
    if sim > 1.0: sim = 1.0
    if sim < 0.0: sim = 0.0
    return sim

def hybrid_score(item: Dict[str, Any], dist: float) -> float:
    s_vec = cosine_from_distance(dist)
    ts = item.get("timestamp") or item.get("updated_at") or item.get("created_at") or ""

```

```

s_rec = _recency_score(ts)
s_ev = _evidence_score(item)
s_cf = _confidence_score(item)
score = 0.45*s_vec + 0.20*s_rec + 0.20*s_ev + 0.15*s_cf
if score > 1.0: score = 1.0
if score < 0.0: score = 0.0
return score

```

24) ai_architect/core/retrieval.py

Python

```

from typing import Dict, Any, List, Optional
from .vectorstore import query_collection, COL_EPISODIC, COL_SEMANTIC
from .memory import load_episode, load_semantic
from .scoring import hybrid_score
from .utils import date_range_filter_ok, new_id, append_jsonl, now_iso
from .config import TRACES_DIR

def _build_where(project: Optional[str], tags: Optional[List[str]], date_from:
Optional[str], date_to: Optional[str]) -> Optional[Dict[str, Any]]:
    # Chroma where filters (simple equality; tags stored as comma string)
    where: Dict[str, Any] = {}
    if project:
        where["project"] = project
    # tags filter can be done post-hoc (easier) to support "contains"
    if date_from or date_to:
        # timestamp range filter post-hoc (Chroma limited); we'll filter after
        pass
    if not where:
        return None
    return where

def retrieve(query: str, top_k: int = 5, project: Optional[str] = None, tags:
Optional[List[str]] = None,
            date_from: Optional[str] = None, date_to: Optional[str] = None, collection:
str = "semantic") -> Dict[str, Any]:
    trace_id = new_id("tr_")
    where = _build_where(project, tags, date_from, date_to)
    if collection not in ("semantic", "episodic"):
        collection = "semantic"
    res = query_collection(collection, query, n_results=top_k*2, where=where)

    results: List[Dict[str, Any]] = []
    ids = res.get("ids", [])
    metas = res.get("metadatas", [])
    docs = res.get("documents", [])
    dists = res.get("distances", [])

    for i in range(len(ids)):
        item_id = ids[i]
        meta = metas[i] if i < len(metas) else {}
        doc = docs[i] if i < len(docs) else ""
        dist = dists[i] if i < len(dists) else 1.0
        # Load full item
        if collection == "semantic":
            try:
                item = load_semantic(item_id)
            except Exception:

```

```

        item = {"id": item_id, "statement": doc, "project": meta.get("project", ""),
"timestamp": meta.get("timestamp", ""), "type": meta.get("type", "fact"), "confidence":
meta.get("confidence", 0.7), "evidence_source_ids": []}
    else:
        try:
            item = load_episode(item_id)
        except Exception:
            item = {"id": item_id, "summary": doc, "project": meta.get("project", ""),
"timestamp": meta.get("timestamp", ""), "evidence": []}
            # Filters post-hoc
            ts = item.get("timestamp") or item.get("updated_at") or item.get("created_at") or
meta.get("timestamp") or ""
            if not date_range_filter_ok(ts, date_from, date_to):
                continue
            if tags:
                item_tags = set(item.get("tags", []))
                if not any(t in item_tags for t in tags):
                    # check meta comma string
                    meta_tags = set(meta.get("tags", "").split(",")) if meta.get("tags") else
set()

                    if not any(t in meta_tags for t in tags):
                        continue
            score = hybrid_score(item, dist)
            results.append({
                "id": item_id,
                "type": item.get("type", "episode" if collection=="episodic" else "semantic"),
                "text": item.get("statement") or item.get("summary") or item.get("title") or
doc,
                "project": item.get("project", meta.get("project", "")),
                "timestamp": ts,
                "score": round(score, 4),
                "evidence": item.get("evidence", []) or [{"type": "episode", "id": eid} for eid
in item.get("evidence_source_ids", [])],
                "collection": collection,
            })
            if len(results) >= top_k:
                break

    # Trace
    trace = {
        "trace_id": trace_id,
        "ts": now_iso(),
        "operation": "retrieve",
        "inputs": {"query": query, "top_k": top_k, "project": project, "tags": tags,
"date_from": date_from, "date_to": date_to, "collection": collection},
        "outputs": {"count": len(results), "results": results},
        "latency_ms": 0,
    }
    append_jsonl(TRACES_DIR / "retrieve.jsonl", trace)
    return {"trace_id": trace_id, "results": results}

```

25) ai_architect/pipelines/ingest.py

Python

```

from typing import Dict, Any
from ..core.utils import now_iso, new_id, redact_text
from ..core.redact import redact_episode
from ..core.memory import save_episode
from ..core.vectorstore import index_episode

```



```

from ..core.utils import append_jsonl
from ..core.config import TRACES_DIR

REQUIRED = ["project", "source_type", "author", "title", "summary", "timestamp"]

def validate_episode(ep: Dict[str, Any]) -> tuple[bool, str]:
    for r in REQUIRED:
        if r not in ep or ep[r] in (None, ""):
            return False, f"Missing required field: {r}"
    # timestamp basic
    try:
        from ..core.utils import ISO
        import datetime as dt
        dt.datetime.strptime(ep["timestamp"], ISO)
    except Exception:
        return False, "timestamp must be ISO8601 UTC (YYYY-MM-DDT%H:%M:%SZ)"
    return True, ""

def build_embedding_text(ep: Dict[str, Any]) -> str:
    parts = [ep.get("title", ""), ep.get("summary", "")]
    for d in ep.get("decisions", []):
        parts.append(d.get("text", ""))
    for a in ep.get("actions", []):
        parts.append(a.get("text", ""))
    for r in ep.get("risks", []):
        parts.append(r.get("text", ""))
    return " \n".join([p for p in parts if p])

def ingest_episode(ep: Dict[str, Any]) -> Dict[str, Any]:
    trace_id = new_id("tr_")
    ok, msg = validate_episode(ep)
    if not ok:
        append_jsonl(TRACES_DIR / "failures.jsonl", {"trace_id": trace_id, "op": "ingest",
"error": msg, "ep": ep})
        return {"ok": False, "error": msg, "trace_id": trace_id}
    # IDs/timestamps
    if "id" not in ep:
        ep["id"] = new_id("ep_")
    if "created_at" not in ep:
        ep["created_at"] = now_iso()
    # Redact
    ep = redact_episode(ep)
    # Embedding text
    ep["embedding_text"] = build_embedding_text(ep)
    # Persist
    save_episode(ep)
    # Index
    try:
        index_episode(ep)
    except Exception as e:
        append_jsonl(TRACES_DIR / "failures.jsonl", {"trace_id": trace_id, "op":
"ingest_index", "error": str(e), "episode_id": ep["id"]})
    # Trace
    append_jsonl(TRACES_DIR / "ingest.jsonl", {"trace_id": trace_id, "ts": now_iso(),
"operation": "ingest", "episode_id": ep["id"], "project": ep.get("project")})
    return {"ok": True, "episode_id": ep["id"], "trace_id": trace_id}

```

26) ai_architect/pipelines/summarize.py

Python

```

from typing import Dict, Any
from ..core.memory import load_episode, save_summary
from ..core.utils import now_iso

def summarize_episode(episode_id: str) -> Dict[str, Any]:
    ep = load_episode(episode_id)
    summary = {
        "episode_id": episode_id,
        "project": ep.get("project"),
        "title": ep.get("title"),
        "summary_text": ep.get("summary", ""),
        "decisions": ep.get("decisions", []),
        "actions": ep.get("actions", []),
        "risks": ep.get("risks", []),
        "created_at": now_iso(),
    }
    save_summary(summary)
    return summary

```

27) ai_architect/pipelines/reflect.py

Python

```

from typing import Dict, Any, List
from ..core.memory import list_episodes, save_reflection
from ..core.utils import now_iso
from collections import Counter

def reflect_project(project: str | None = None, days: int = 30) -> Dict[str, Any]:
    eps = list_episodes(project=project)
    # simple heuristic reflection (demo)
    tag_counter = Counter()
    risk_counter = Counter()
    decision_themes: List[str] = []
    for ep in eps:
        for t in ep.get("tags", []):
            tag_counter[t] += 1
        for r in ep.get("risks", []):
            risk_counter[r.get("text", "")] += 1
        for d in ep.get("decisions", []):
            txt = d.get("text", "")
            if "autentic" in txt.lower() or "auth" in txt.lower():
                decision_themes.append("Autenticación/autorización recurrente")
                break
    patterns = [f"Top tag: {tag} ({cnt})" for tag, cnt in tag_counter.most_common(5)]
    if decision_themes:
        patterns.append("Tema recurrente: " + "; ".join(sorted(set(decision_themes))))
    contradictions: List[str] = []
    ref = {
        "scope": {"project": project, "days": days},
        "patterns": patterns,
        "contradictions": contradictions,
        "hypotheses": ["Revisar decisiones de auth si aparecen >2 veces en 30 días."],
        "created_at": now_iso(),
    }
    save_reflection(ref)
    return ref

```

28) ai_architect/pipelines/factcheck.py

Python

```
from typing import Dict, Any, List
from ..core.memory import load_episode, list_semantic
from ..core.utils import now_iso, new_id
from ..core.config import TRACES_DIR
from ..core.utils import append_jsonl

def factcheck_episode(episode_id: str) -> Dict[str, Any]:
    trace_id = new_id("tr_")
    ep = load_episode(episode_id)
    project = ep.get("project")
    # Buscar decisiones similares en semántico
    sems = list_semantic(project=project, type_="decision")
    matched: List[Dict[str, Any]] = []
    contradictions: List[Dict[str, Any]] = []
    for d in ep.get("decisions", []):
        dtext = d.get("text", "").lower()
        for s in sems:
            stext = s.get("statement", "").lower()
            if dtext and stext and (dtext in stext or stext in dtext or
len(set(dtext.split()) & set(stext.split())) >= 3):
                matched.append({"semantic_id": s.get("id"), "statement":
s.get("statement")})
                # simple contradicción: contiene "no usar" vs "usar"
                if ("no usar" in dtext and "usar" in stext) or ("no usar" in stext and
"usar" in dtext):
                    contradictions.append({"semantic_id": s.get("id"), "reason": "Posible
contradicción (usar/no usar)"})
        confidence = 0.8
        if contradictions:
            confidence = max(0.3, confidence - 0.3)
        if len(ep.get("evidence", [])) == 0:
            confidence = max(0.2, confidence - 0.2)
        res = {
            "trace_id": trace_id,
            "episode_id": episode_id,
            "ok": len(contradictions) == 0,
            "confidence": round(confidence, 2),
            "matched_semantic_ids": [m["semantic_id"] for m in matched],
            "contradictions": contradictions,
            "actions": ["Revisar evidencia y alinhar decisión"] if contradictions else
["Aprobado para consolidación"],
            "ts": now_iso(),
        }
        append_jsonl(TRACES_DIR / "factcheck.jsonl", res)
    return res
```

29) ai_architect/pipelines/consolidate.py

Python

```
from typing import Dict, Any, List
from ..core.memory import load_episode, save_semantic, list_semantic
from ..core.vectorstore import index_semantic
from ..core.utils import now_iso, new_id
```

```

from ..core.config import TRACES_DIR
from ..core.utils import append_jsonl
from .summarize import summarize_episode
from .factcheck import factcheck_episode

def consolidate_project(project: str | None = None) -> Dict[str, Any]:
    trace_id = new_id("tr_")
    # Listar episodios
    from ..core.memory import list_episodes
    eps = list_episodes(project=project)
    promotions = 0
    contradictions = 0
    for ep in eps:
        # Asegurar resumen
        try:
            summarize_episode(ep["id"])
        except Exception:
            pass
        # Factcheck
        fc = factcheck_episode(ep["id"])
        if not fc["ok"]:
            contradictions += 1
            continue
        if fc["confidence"] < 0.6:
            continue
        # Promover decisiones a semántico (demo: primera decisión)
        decisions = ep.get("decisions", [])
        if not decisions:
            continue
        d = decisions[0]
        # Evitar duplicado exacto simple
        sems = list_semantic(project=ep.get("project"), type_="decision")
        dup = False
        dtext = d.get("text", "").lower()
        for s in sems:
            if s.get("statement", "").lower() == dtext:
                dup = True
                # actualizar evidencia/confianza
                ev = set(s.get("evidence_source_ids", []))
                ev.add(ep["id"])
                s["evidence_source_ids"] = list(ev)
                s["confidence"] = min(1.0, s.get("confidence", 0.6) + 0.05)
                s["updated_at"] = now_iso()
                save_semantic(s)
                index_semantic(s)
                break
        if dup:
            continue
        # Crear semántico
        item = {
            "type": "decision",
            "project": ep.get("project"),
            "statement": d.get("text"),
            "confidence": fc["confidence"],
            "evidence_source_ids": [ep["id"]],
            "contradictions": fc.get("contradictions", []),
            "tags": ep.get("tags", []),
            "embedding_text": d.get("text"),
        }
       .pth = save_semantic(item)
        index_semantic(item)
        promotions += 1

```

```

    res = {"trace_id": trace_id, "ts": now_iso(), "operation": "consolidate", "project":
project, "promotions": promotions, "contradictions": contradictions}
    append_jsonl(TRACES_DIR / "consolidate.jsonl", res)
    return res

```

30) ai_architect/pipelines/evaluate.py

Python

```

from typing import Dict, Any, List
from ..core.retrieval import retrieve
from ..core.utils import now_iso, new_id
from ..core.config import EVALS_DIR
from ..core.utils import append_jsonl

def evaluate_query(query: str, project: str | None = None, top_k: int = 5, collection: str
= "semantic") -> Dict[str, Any]:
    trace_id = new_id("tr_")
    r = retrieve(query=query, top_k=top_k, project=project, collection=collection)
    results = r.get("results", [])
    # Métricas simples (demo: heurísticas; puedes reemplazar por calificación humana/LLM)
    relevance = []
    has_evidence = []
    halluc_risk = []
    for res in results:
        # Heurística: score alto => relevance alta
        rel = min(1.0, res.get("score", 0.0) + 0.2)
        ev = len(res.get("evidence", [])) > 0
        hr = 1.0 - rel
        if ev:
            hr = max(0.0, hr - 0.3)
        relevance.append(round(rel, 2))
        has_evidence.append(ev)
        halluc_risk.append(round(hr, 2))
    eval_rec = {
        "trace_id": trace_id,
        "ts": now_iso(),
        "query": query,
        "project": project,
        "collection": collection,
        "retrieved_ids": [x["id"] for x in results],
        "scores": [x["score"] for x in results],
        "relevance": relevance,
        "has_evidence": has_evidence,
        "hallucination_risk": halluc_risk,
        "notes": "Heurístico (demo). Sustituir por evaluación humana o LLM judge.",
    }
    append_jsonl(EVALS_DIR / "evaluations.jsonl", eval_rec)
    return eval_rec

```

31) ai_architect/pipelines/export.py

Python

```

import zipfile, datetime as dt
from pathlib import Path
from ..core.config import MEMORY_ROOT, LOG_DIR, CHROMA_DIR, ROOT

```

```

from ..core.utils import ensure_dir

def export_all(out_dir: str | None = None) -> Path:
    ts = dt.datetime.utcnow().strftime("%Y-%m-%d_%H-%M-%SZ")
    if out_dir is None:
        out_dir = ROOT / ".ai_memory" / "exports"
    ensure_dir(Path(out_dir))
    zip_path = Path(out_dir) / f"{ts}.zip"
    with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as z:
        for p in [MEMORY_ROOT, LOG_DIR, CHROMA_DIR]:
            if p.exists():
                for f in p.rglob("*"):
                    if f.is_file():
                        z.write(f, f.relative_to(ROOT))
    return zip_path

```

32) ai_architect/core/mcp_server.py (FastAPI)

Python

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List, Optional
from ..pipelines.ingest import ingest_episode
from ..core.retrieval import retrieve
from ..pipelines consolidate import consolidate_project
from ..pipelines.evaluate import evaluate_query

app = FastAPI(title="AI Knowledge MCP", version="0.1.0")

class IngestReq(BaseModel):
    episode: dict

class RetrieveReq(BaseModel):
    query: str
    top_k: int = 5
    project: Optional[str] = None
    tags: Optional[List[str]] = None
    date_from: Optional[str] = None
    date_to: Optional[str] = None
    collection: str = "semantic"

class ConsolidateReq(BaseModel):
    project: Optional[str] = None

class EvaluateReq(BaseModel):
    query: str
    project: Optional[str] = None
    top_k: int = 5
    collection: str = "semantic"

@app.get("/health")
def health():
    return {"ok": True}

@app.post("/ingest")
def ingest(req: IngestReq):
    res = ingest_episode(req.episode)
    if not res.get("ok"):
        raise HTTPException(status_code=400, detail=res)

```

```

        return res

@app.post("/retrieve")
def retrieve_ep(req: RetrieveReq):
    res = retrieve(req.query, req.top_k, req.project, req.tags, req.date_from, req.date_to,
req.collection)
    return res

@app.post("/consolidate")
def consolidate(req: ConsolidateReq):
    res = consolidate_project(req.project)
    return res

@app.post("/evaluate")
def evaluate(req: EvaluateReq):
    res = evaluate_query(req.query, req.project, req.top_k, req.collection)
    return res

```



Scripts CLI / Demo listos para ejecutar

33) make_demo.py (end-to-end)

Python

```

from ai_architect.pipelines.ingest import ingest_episode
from ai_architect.pipelines.summarize import summarize_episode
from ai_architect.pipelines.reflect import reflect_project
from ai_architect.pipelines.factcheck import factcheck_episode
from ai_architect.pipelines.consolidate import consolidate_project
from ai_architect.core.retrieval import retrieve
from ai_architect.pipelines.evaluate import evaluate_query
from ai_architect.core.utils import now_iso
from pathlib import Path
import json

def main():
    print("🚀 Running demo...")

    # Episodio 1
    ep1 = {
        "project": "demo-auth",
        "source_type": "meeting",
        "author": "ana",
        "title": "Decisión de autenticación",
        "summary": "El equipo decide usar JWT con refresh tokens rotativos y revocación por
lista negra en Redis.",
        "timestamp": now_iso(),
        "decisions": [
            {"text": "Usar JWT con refresh tokens rotativos", "owner": "ana"},
            {"text": "Implementar revocación por lista negra en Redis", "owner": "luis"},
        ],
        "actions": [{"text": "Implementar servicio auth", "owner": "luis", "due_date":
"2026-05-30T00:00:00Z"}],
        "risks": [{"text": "Riesgo de gestión de claves JWT", "severity": "high"}],
        "evidence": [{"type": "doc", "url_or_path": "confluence://auth/ADR-01", "excerpt":
"ADR-01 aprueba JWT + refresh rotativo"}],
        "tags": ["auth", "seguridad"],

```

```

    }
    r1 = ingest_episode(ep1)
    print("Ingest ep1:", r1)

    # Episodio 2 (repetición temática)
    ep2 = {
        "project": "demo-auth",
        "source_type": "pr",
        "author": "luis",
        "title": "Implementación auth service",
        "summary": "Se mergea implementación de JWT con refresh rotativos y blacklist
Redis.",
        "timestamp": now_iso(),
        "decisions": [
            {"text": "Usar JWT con refresh tokens rotativos", "owner": "luis"},
        ],
        "actions": [{"text": "Configurar rotación y revocación", "owner": "luis"}],
        "risks": [{"text": "Fuga de refresh token", "severity": "medium"}],
        "evidence": [{"type": "pr", "url_or_path": "github.com/org/repo/pull/42"},
    "excerpt": "PR #42 implementa refresh rotativo"}],
        "tags": ["auth", "backend"],
    }
    r2 = ingest_episode(ep2)
    print("Ingest ep2:", r2)

    # Resumen + reflexión + factcheck
    summarize_episode(r1["episode_id"])
    summarize_episode(r2["episode_id"])
    reflect_project("demo-auth")
    factcheck_episode(r1["episode_id"])
    factcheck_episode(r2["episode_id"])

    # Consolidación (promueve decisión a semántico)
    cons = consolidate_project("demo-auth")
    print("Consolidate:", cons)

    # Retrieval
    ret = retrieve(query="decisión de autenticación JWT", top_k=3, project="demo-auth",
collection="semantic")
    print("Retrieve (semantic):", json.dumps(ret, ensure_ascii=False, indent=2))

    # Evaluación
    ev = evaluate_query(query="decisión de autenticación JWT", project="demo-auth",
top_k=3, collection="semantic")
    print("Evaluate:", json.dumps(ev, ensure_ascii=False, indent=2))

    print("✅ Demo completada. Revisa: memory/episodic/, memory/semantic/,
memory/summaries/, memory/reflections/, logs/traces/, logs/evaluations/,
.ai_memory/chroma/")

if __name__ == "__main__":
    main()

```

34) ingest_cli.py

Python

```

import typer, json
from pathlib import Path
from ai_architect.pipelines.ingest import ingest_episode

```



```

app = typer.Typer()

@app.command()
def main(file: str):
    p = Path(file)
    data = json.loads(p.read_text(encoding="utf-8"))
    res = ingest_episode(data)
    print(json.dumps(res, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    app()

```

35) retrieve_cli.py

Python

```

import typer, json
from typing import Optional, List
from ai_architect.core.retrieval import retrieve

app = typer.Typer()

@app.command()
def main(query: str, top_k: int = 5, project: Optional[str] = None, collection: str = "semantic", tags: Optional[str] = None, date_from: Optional[str] = None, date_to: Optional[str] = None):
    tag_list = tags.split(",") if tags else None
    res = retrieve(query=query, top_k=top_k, project=project, tags=tag_list, date_from=date_from, date_to=date_to, collection=collection)
    print(json.dumps(res, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    app()

```

36) consolidate_cli.py

Python

```

import typer, json
from typing import Optional
from ai_architect.pipelines.consolidate import consolidate_project

app = typer.Typer()

@app.command()
def main(project: Optional[str] = None):
    res = consolidate_project(project)
    print(json.dumps(res, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    app()

```

37) evaluate_cli.py

Python

```
import typer, json
from typing import Optional
from ai_architect.pipelines.evaluate import evaluate_query

app = typer.Typer()

@app.command()
def main(query: str, project: Optional[str] = None, top_k: int = 5, collection: str =
"semantic"):
    res = evaluate_query(query=query, project=project, top_k=top_k, collection=collection)
    print(json.dumps(res, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    app()
```



Verificación rápida (para confirmar que todo funciona)

Después de `python make_demo.py` deberías ver:

- `memory/episodic/ep_*.json` (2 archivos)
- `memory/summaries/sum_*.json` (2)
- `memory/semantic/sem_*.json` (≥1 decisión promovida, p.ej. “Usar JWT con refresh tokens rotativos”)
- `.ai_memory/chroma/` con datos (colecciones `episodic/semantic`)
- `logs/traces/ingest.jsonl`, `factcheck.jsonl`, `consolidate.jsonl`, `retrieve.jsonl`
- `logs/evaluations/evaluations.jsonl`



Notas importantes (para producción / siguiente nivel)

- Autenticación MCP: en prod añade API key/JWT, rate limiting, CORS restringido, validación estricta de esquemas (usa Pydantic models también en ingest/retrieve).

- Embeddings reales: cambia `EMBEDDING_PROVIDER=openai` y define `OPENAI_API_KEY`. Re-indexar no es obligatorio si mantienes misma función, pero conviene reindexar si cambias modelo/dimensión.
- Búsqueda híbrida avanzada: añade filtros por `tags` *dentro* de Chroma (usa `where: {"$and": [...]}`) o mantén post-hoc (ya implementado). Para grandes volúmenes, prefiere filtros nativos.
- Versionado semántico: cuando una decisión cambia, crea nuevo `sem_*.json` con `supersedes/superseded_by` y marca el anterior `status="deprecated"`.
- PII: el `redact.py` es básico; amplía patrones según tu dominio y añade tests.

Si quieres, también puedo generarte: (1) tests unitarios con `pytest`, (2) Dockerfile + docker-compose (Chroma opcional + API), (3) un tablero simple en Markdown/CSV para métricas de `logs/evaluations/`, o (4) adaptarlo a tu herramienta concreta (OpenCode/Cline/Aider/Continue) con ejemplos de tool-calls JSON.